

[← SCENE DEMO](#)[UVW EXPLAINER](#)[SOURCE ON GITHUB](#)

Proposal: World-coord-keyed canonical encoding via a bidirectional UVW↔RGB atlas

Targeted at: [nv-tlabs/lyra](#), Lyra 2.0 **Author:** MiLO + Opie (github.com/MiLO83/DownToEarth) **Status:** Architectural proposal — research / discussion, not a PR **License of this document:** MIT. Code references in this doc are from the Apache-2.0-licensed Lyra source tree.

In plain English (the wins, the losses, the gist)

The setup

Lyra 2's current way: keep a separate photo album for every photo it's ever taken of your house. If it photographs the kitchen from 5 angles, that's 5 album entries. To find "what's the kitchen wall?" it has to flip through every album looking for the right one.

Our proposed way: keep a single floor-plan map of the house. Every spot on the map has a unique address. No matter how many photos you take of the kitchen wall, it stays in one place on the map. To find it, glance at the map once.

That's the whole idea. Everything in the technical sections below follows from that.

Wins (why this could be worth it)

- 1. No more memory bloat from looking at the same thing twice.** The camera can walk around a room and see the same wall fifty times. Old way: fifty new entries. New way: the wall's spot on the map gets refreshed in place. Memory stays the same size once the house is fully mapped.
- 2. Finding stuff is one step, not fifty.** "Where's the kitchen wall?" Old way: flip through every photo album. New way: read the address off the map. The bigger the cache gets, the bigger the gap.
- 3. Same place, same name.** The model never has to figure out "wait, is this kitchen wall from album 3 the same as kitchen wall from album 7?" The map address IS the identity. One thing, one name, no ambiguity.
- 4. You can look at the map and read it.** Today the photo-album labels are codes like "album 47, photo 12, pixel (143, 89)." The new map's labels are actual map coordinates — you can literally print the map and a person can read it. Debugging stops being a guessing game.
- 5. Neighbours stay neighbours.** On the map, two spots physically next to each other in the real world are also physically next to each other on the page. Your GPU loves this — it can grab a whole little chunk of map at once, instead of running off to fetch random photo albums from across storage. Modest speed-up, basically free.

Losses (why this isn't a slam dunk)

- 1. The map has a grid; tiny details below grid size disappear.** At the cheapest setting (1 byte per axis), each map square is about 1 cm. Two things half a centimetre apart get squashed into the same square. For most architecture-scale stuff

this is fine. For very tiny detail you'd need a finer map (more bytes per axis), which costs more memory.

→ SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

2. **You can't make a single map of the whole world.** A bedroom-sized map works great. A city-sized map needs to be folders of regional maps. A continent-sized map needs folders of folders. Lyra 2's specialty is "walk-forever" scenes — those need the folder-of-maps version, which is more engineering work. We sketched how but didn't build it.
3. **The model has to learn a new language.** Today's Lyra 2 has been trained to read photo-album labels. To switch to map addresses, the model needs to be re-taught — that's roughly six weeks of training on 64 of NVIDIA's most expensive GPUs. We can't do this; only NVIDIA can. So this is a "if you're already planning to retrain, consider folding this in" proposal, not a drop-in patch.
4. **The map doesn't remember WHEN you mapped each spot.** Photo albums have dates. "This album was from yesterday, that one was from last month — trust yesterday's more." The map just shows the latest state of each spot. To get the "when" back you'd attach a little timestamp byte to each map cell (we have a spare byte in our 4-byte-per-cell budget; it's already documented). Not lost, just an extra design choice.
5. **It only helps if there's stuff to map.** For a single static photo with no exploration, both approaches do basically the same thing. The wins kick in as the camera moves around and the same locations get re-observed — which IS Lyra 2's actual job, so it's the right shape, but worth being honest: a one-shot single-frame use-case sees almost no benefit.

The honest one-liner

Lyra 2 already computes the world-coords of every pixel — it just throws that information into a photo-album-style index. Re-indexing into a map-style atlas is the architectural shift; it's cleaner, faster, and bounded by scene size instead of frame count, but it costs a retrain and the addressable world is now finite (or hierarchical). That's the whole trade.

Summary

Lyra 2's canonical-coord conditioning is **image-space-and-frame-indexed**: each pixel of the warped conditioning image encodes `(u_normalized, v_normalized, frame_slot_idx)`. The cross-attention then treats these as keys into a *frame-keyed* history of latents. This works, but two consequences fall out:

1. **Multiple past frames seeing the same 3D point produce different canonical-coord values.** Geometric correspondence is implicit and has to be re-learned at every attention layer.
2. **History size grows with time, not scene complexity.** A frame that re-observes already-known geometry still occupies a full slot.

Lyra 2's `Sparse3DCache` already computes per-pixel world coordinates (via `unproject_points` at `lyra2_model.py:2534`), so the world-coord information is present — it's simply not used as the canonical-coord key.

This proposal: **encode the canonical-coord image as quantized world coords (packed into RGB bytes via a bijective tile layout), and back it with a world-coord-keyed atlas instead of a frame-keyed cache.** Same warp machinery, different output encoding, different storage layout. Below: the mechanics, the wins, the trade-offs, and an estimated porting cost.

The current Lyra 2 mechanism (with citations)

I'll keep this brief — the paper covers it well — but anchor each claim to source.

Canonical-coord construction

← SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

`_build_canonical_spatial_coords` (`lyra2_model.py:1525-1545`):

```
xs = torch.linspace(-1.0, 1.0, W, device=device, dtype=dtype)
ys = torch.linspace(-1.0, 1.0, H, device=device, dtype=dtype)
yy, xx = torch.meshgrid(ys, xs, indexing="ij")
base_xy = torch.stack([xx, yy], dim=0) # [2, H, W]
...
zs = torch.linspace(-1.0, 1.0, num_spatial_hist, device=device, dtype=dtype)
z = zs.view(num_spatial_hist, 1, 1, 1).expand(num_spatial_hist, 1, H, W)
coords = torch.cat([base_xy, z], dim=1) # [N, 3, H, W]
```

So each pixel of slot `n` starts life as `(u_normalized, v_normalized, z_n)` where `z_n` is the slot's position along `[-1, 1]`. These get forward-warped via `forward_warp_multiframes` (`forward_warp_utils_pytorch.py:57`) through past camera poses + depths into the target view. The output of the warp is the canonical-coord image fed to the DiT as conditioning.

Storage and retrieval

`Sparse3DCache` (`lyra2_model.py:2488`) keeps:

```
self._world_points: list[torch.Tensor] = [] # each: [B, H', W', 3]
self._latent_indices: list[int] = []
self._frame_ids: list[int] = []
self._depths: list[torch.Tensor] = []
self._w2cs: list[torch.Tensor] = []
self._Ks: list[torch.Tensor] = []
self._rgbs: dict[int, torch.Tensor] = {}
```

`add()` calls `unproject_points` to compute world coords per pixel (line 2534), then stores those alongside the depth/w2c/K and the RGB. **The world coords are computed and stored.** Retrieval (`Sparse3DCache.retrieve`) selects past frames by visibility overlap with the target view and returns their latent indices for cross-attention.

So: **frame-keyed cache, image-space-plus-frame canonical-coords.**

The proposed alternative

Encoding change

Replace the `(u_norm, v_norm, frame_slot)` encoding with **quantized world coords packed into RGB bytes**:

```
# Quantize world coords to [0, 2^B - 1] per axis (B = bits per channel).
# For B = 8 (RGBA8): 256 levels per axis = 16.7M unique 3D positions in a
# bounded world. For B = 16 (RGBA16UI): 65,536 levels = 281T positions.
def world_to_canonical_rgb(world_xyz: Tensor, world_min: Tensor,
                           world_max: Tensor, bits: int = 8) -> Tensor:
    n = (1 << bits) - 1
    normed = (world_xyz - world_min) / (world_max - world_min) # [0, 1]
    return (normed.clamp(0, 1) * n).to(torch.uint8 if bits == 8 else torch.uint16)
```

This is byte-perfect bijective: the bytes in the canonical image are the world coords (modulo a known world_min/max transform). The current warp machinery transports them correctly → `forward_warp_multiframes` doesn't care whether `frame1` is RGB, normalized image coords, or quantized world coords; it just resamples per the depth + camera transform.

Storage change

Replace the frame-keyed `Sparse3DCache` with a **world-coord-keyed atlas** addressed via a 2D tile layout (the bijection: $256^3 \rightarrow 4096^2$ for a single- texture-array fit, hierarchical sparse beyond that):

```
# Pack (u, v, w) ∈ [0, 256]^3 into a 2D atlas position via a 16×16 tile grid.
# Each tile is one full w-slice of size 256×256.
def voxel_to_atlas(u: int, v: int, w: int) -> tuple[int, int]:
    return ((w & 15) << 8) | u, ((w >> 4) << 8) | v
```

The atlas stores whatever Lyra 2 currently stores per past frame — latent feature vectors, RGB, or both — but indexed by 3D position instead of by `(frame_id, pixel_position)`. When the same 3D point gets observed by multiple frames, the new write **strengthens or replaces** the existing slot rather than adding a new frame entry.

This bijection is documented + verified in `voxgaussian/pipeline/uvw_atlas.py` with an exhaustive 16.7M-pair round-trip test. The identity mapping (atlas-position ↔ canonical-pass RGB) is *mathematically inherited* and costs zero VRAM — only the payload uses storage.

What changes in the cross-attention

Currently cross-attention is over frame latents, keyed by the warped `(u_norm, v_norm, frame_slot)`. With the proposal, cross-attention is over the atlas, keyed by the warped `(world_x, world_y, world_z)`. The attention math is unchanged; only the storage layout changes.

Why this is interesting (the wins)

1. **Geometric correspondence becomes structural, not learned.** A 3D point seen by frames 3, 7, and 12 has *one* key — its world coord — instead of three different `(u, v, frame_slot)` triples that the model must learn to treat as equivalent. This should reduce the inductive load on the cross-attention.
2. **Memory scales with scene complexity, not time.** Re-observing known geometry reinforces existing atlas slots; it doesn't allocate new frame slots. For long camera trajectories that revisit already- explored regions (which Lyra 2's 90 m worlds do, by design), this bounds memory growth.
3. **O(1) "is this 3D point known?" lookup.** Currently `Sparse3DCache.retrieve` scores all candidate frames by visibility overlap (see `lyra2_model.py:2737-2832`). With a world-coord atlas, the question becomes a texture sample at the warped coord — single instruction.
4. **Spatial locality on GPU.** A tile-layout (or Hilbert-curve) bijection preserves 3D-neighbour adjacency in the 2D atlas, so cache prefetch helps for the typical "raymarch-y" access pattern.
5. **Debuggable conditioning.** Visualizing the canonical-coord image shows you exactly what the model sees: pixel colors *are* coords. No inverse-projection step to interpret it.

Trade-offs (the catches)

SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

1. **Bounded vs. unbounded scenes.** 256^3 fits in 4096^2 (16.7 M atlas slots, $\approx$ 2.5 m world at 1 cm cells). Lyra 2's 90 m+ worlds need a hierarchical sparse-tiled version — atlas grows octree-style as the camera moves into new regions. Doable but more engineering. RGBA16 pushes the bounded reach to $\approx$ 655 m at 1 cm cells.
2. **Retraining required.** The model has learned to read `(u, v, frame_slot)`. Adopting world-coord encoding changes its inputs — at minimum the conditioning head needs fine-tuning, more likely full retraining. Cost-prohibitive to attempt this without NVIDIA's training infrastructure.
3. **Loss of temporal information.** Frame index carries "when did we see this" signal that can be useful for weighting recent observations higher (drift correction). To preserve this in the atlas-keyed scheme, attach a `last_updated_iter` byte to each atlas slot — uses one of the 4 RGBA bytes the address doesn't need for spatial identity.
4. **Quantization error.** World coords get bucketed into the atlas resolution. At 8-bit per axis (256^3), a 2.5 m world has 1 cm cells — probably fine for the kind of geometry Lyra 2 works with, but noticeable for very small detail. RGBA16 fixes this entirely.
5. **Doesn't help the unbounded-walk case directly.** Lyra 2's spec is *autoregressive long-horizon generation*. For each new chunk, the camera is somewhere new and we need to extend the world. A pure bounded atlas saturates; a hierarchical one is required for the full spec to be preserved.

Performance cross-reference (before vs. after)

All numbers calibrated to Lyra 2's documented config: **832 × 480 resolution, 80 frames per chunk, num_spatial_hist = 5, Sparse3DCache downsample = 4**. * = calculated from the codebase. † = estimated. ‡ = qualitative.

Dimension	Current (Lyra 2 frame-keyed)	Proposed (world-coord atlas)	Delta
Canonical-coord image size, per chunk	$5 \times 3 \times 480 \times 832 \times \text{fp16} = 11.4 \text{ MB} *$	RGBA8: 5.71 MB / RGBA16UI: 11.4 MB / RGBA32F: 22.9 MB *	RGBA8: 2× smaller than fp16 / 4× smaller than fp32; RGBA16: parity
Cache memory: <code>_world_points</code> , after 80 frames	$80 \times (480/4) \times (832/4) \times 3 \times \text{fp32} = 24 \text{ MB} *$	$256^3 \text{ atlas} \times 4 \text{ B (RGBA8)} = \mathbf{64 \text{ MB}}$ fixed ($1024^3 \rightarrow \sim 4 \text{ GB}$ texture array) †	Larger up front, flat with time vs. linear
Cache memory: with <code>store_values=True</code> (full depth + w2c + K + RGB latents)	$80 \times (480 \times 832 \times \text{fp32 depth} + \text{intrinsics} + \text{latents}) \approx \mathbf{130 \text{ MB} + \text{frame latents} *}$	Atlas + per-slot payload bytes; no per-frame redundant storage. $\sim 64 \text{ MB}$ to $\sim 1 \text{ GB}$ scene-dependent. †	Scene-complexity-bound, not time-bound
Memory growth on revisits (camera re-enters region already seen)	Linear: every frame adds a slot whether new or not	Flat: same world-coord slot gets reinforced, no new allocation	O(time) → O(scene)

Dimension	Current (Lyra 2) frame-keyed	Proposed (world-coord atlas) UVW EXPLAINER	Delta
<code>Sparse3DCache.retrieve()</code> cost per target view	Scores all N candidate slots by visibility overlap; iterate + sort	Single 4-instruction warp + texture sample per target pixel	O(N) → O(1) per pixel
Cross-attention KV count	num_spatial_hist (5) × tokens-per-frame	num_spatial_hist (5) × tokens-per-frame (unchanged — same model arch)	Same
Distinct keys per 3D point across views	One per past-frame-that-saw-it; the model must learn equivalence	Exactly one (the quantized world coord)	Inductive load ↓
Cache write atomicity (multi-view fusion)	Sequential per-frame appends; merge logic in <code>retrieve()</code>	Atomic increment / max into atlas slots; GPU <code>atomicAdd</code> on <code>R32_UINT</code>	Trivially parallel
GPU cache locality (spatial neighbours)	Random — frames stored independently; access pattern depends on the warp permutation	Preserved — 16×16 tile layout keeps 3D neighbours within ~256 px in the 2D atlas	Texture-cache friendly
Quantization error	None (fp32 world coords retained internally)	1 LSB per axis: 1 cm at 256 ³ /2.5 m world (RGBA8), <0.1 mm at RGBA16. None at RGBA32 *	RGBA8: low / RGBA16: negligible
Disocclusion-hole rendering quality	Cleanly black where no past frame saw the pixel	Same — atlas lookup returns (0,0,0) where no slot has been written	Identical
Long-horizon (Lyra 2 spec, 90 m walk-through)	Frame count grows linearly with trajectory length; retrieval slows	Requires hierarchical sparse atlas — bounded variant saturates beyond world_max ‡	Open engineering question, advantage <i>if</i> sparse-tiled is built
Conditioning interpretability (visualize the canonical image)	Hard to read (normalized image-coords + frame idx)	RGB literally encodes (u, v, w) — visual debug is the bytes ‡	Strict win for development
Code surface area (LoC change)	—	<code>_build_canonical_spatial_coords</code> rewrite (50 LoC) + <code>Sparse3DCache</code> dual-key mode (100 LoC) + new <code>uvw_atlas.py</code> (150 LoC): **300 LoC net add.** †	Small
Training-compat impact	—	Full retrain or fine-tune the conditioning head: model expects different statistics on the	Significant — needs NVIDIA

Dimension	Current (Lyra 2) — SCENE DEMO ? frame-keyed	Proposed (world-coord atlas) UVW EXPLAINER	SOURCE ON GITHUB	Delta
		input channels. ‡		training infra
Inference latency (per 80-frame step)	~194 s on GB200 (full), ~15 s (DMD-distilled) * (from paper)	Estimate: ~1–3 % reduction from cheaper canonical-coord pass + O(1) retrieval; possible 5–10 % reduction at long-horizon revisits where current retrieve() dominates ‡		Modest, possibly meaningful at scale

Byte-length comparison (per-channel precision)

Held constant: Lyra 2 res (832 × 480), 5 spatial slots, 3 coord channels, a single inference chunk. Atlas dims assume the maximum single-texture limit (16384²) at each format; beyond that you need a texture array or sparse-tiled scheme.

Format	Bytes / axis	Max axis val	World @ 1 cm cells	World @ 1 mm cells	Canonical img (5×3×480×832)	Single-texture atlas footprint	Identity flavour	GPU support
RGBA8 (uint8)	1	256	2.56 m	25.6 cm	5.71 MB	16384 ² × 4 B = 1 GB	byte = coord (exact)	universal
RGBA16UI (uint16)	2	65,536	655 m	65.5 m	11.4 MB	16384 ² × 8 B = 2 GB	uint16 = coord (exact)	WebGL2 + EXT_color_buffer_integer
RGBA16F (half)	2	1,024 (mantissa)	10.2 m	1.02 m	11.4 MB	16384 ² × 8 B = 2 GB	float ≈ coord (exact ≤ 2 ¹⁰ , lossy after)	WebGL2 native
RGBA32F (float32)	4	16.7M (exact)	167 km	16.7 km	22.9 MB	16384 ² × 16 B = 4 GB	float = coord (exact ≤ 2 ²⁴)	WebGL2 native
RGBA32UI (uint32)	4	4.29B	42,000 km	4,200 km	22.9 MB	16384 ² × 16 B = 4 GB	uint32 = coord (exact)	WebGL2 + EXT_color_buffer_integer

Same dimensions vs. Lyra 2's existing fp16 / fp32 canonical-coord image:

Lyra 2 today	Proposed equivalent	Image size delta	Identity delta
Current: fp16 (2 B/axis)	RGBA8 (1 B/axis)	2× smaller image	Quantize: 1 cm @ 2.5 m world, but byte-perfect vs. fp16's float-rounding

Lyra 2 today	Proposed equivalent	Image size delta	Identity delta
		← SCENE DEMO	UVW EXPLAINER
			SOURCE ON GITHUB
Current: fp16 (2 B/axis)	RGBA16UI (2 B/axis)	Same size	byte-perfect vs. float-rounding; 655 m @ 1 cm world available
Current: fp32 (4 B/axis)	RGBA16UI (2 B/axis)	2× smaller	byte-perfect; 655 m @ 1 cm reach
Current: fp32 (4 B/axis)	RGBA32UI (4 B/axis)	Same size	byte-perfect vs. float-rounding; effectively unbounded

Recommendation by Lyra 2 use-case

Scene scale	Recommended format	Atlas storage	Notes
≤ 2.5 m (single room / character)	RGBA8	64 MB at 256 ³	Plenty for the bounded variant; pairs with single 4096 ² texture, the original DownToEarth target
2.5–655 m (city block, building cluster)	RGBA16UI	1–2 GB texture array	Matches Lyra 2's typical 90 m world spec at sub-cm precision
655 m – 167 km (urban / regional)	RGBA32F	Sparse-tiled, scene-dependent	Mantissa exact to 2 ²⁴ ; lose strict byte-identity outside that
Unbounded (continent+)	Hierarchical sparse + RGBA32UI	Octree-style page table	Byte-identity preserved within each leaf tile ; cross-tile is one indirection

The format choice is **independent** of the architectural proposal — even at RGBA8 you get the structural wins (one key per 3D point, O(1) retrieval, memory bounded by scene-not-time). The byte width just sets the addressable world size.

What this table is *not*

- **Not a quality-score comparison.** SSIM, LPIPS, FID — none of these can be predicted from the architecture change alone. They require a retrain to measure. The above is purely the *computational* and *memory* analysis.
- **Not a benchmark.** Numbers above are calculated or estimated from the codebase + the paper. No actual runs were performed (the proposers don't have GB200s).
- **Not a free win.** The retraining cost is the elephant in the room. The argument is the per-inference math is cleaner and the memory growth on long trajectories is bounded — *if* a retrain is in the budget anyway, this is the kind of architectural shift to fold in.

Concrete porting sketch (if anyone wants to try)

Three files change, roughly:

1. lyra_2/_src/models/lyra2_model.py::_build_canonical_spatial_coords

```
def _build_canonical_spatial_coords(H, W, num_spatial_hist, device, dtype, *,
                                    world_min, world_max, bits=8):
```

```
# Generate a per-pixel world-coord initialization that the warp will
# then transport. Each SPR SCENE DEMO still UVW:EXPLAINER has SOURCE ON GITHUB
# axis — but the axis is "depth tag" not "frame slot" so warps from
# different past frames that landed on the same 3D point produce the
# same canonical RGB.
xs = torch.linspace(world_min[0], world_max[0], W, ...)
ys = torch.linspace(world_min[1], world_max[1], H, ...)
...
return coords # [N, 3, H, W] in WORLD units (will be re-quantized post-warp)
```

After `forward_warp_multiframes` runs, quantize the warped output to `uint8 / uint16` via `world_to_canonical_rgb`. The downstream `_pixelshuffle_hw_to_latent` (line 1568) is dtype-agnostic.

2. `lyra_2/_src/models/lyra2_model.py::Sparse3DCache`

Add a world-coord-keyed mode alongside the existing frame-keyed one. A single `dict[atlas_pos, payload]` (or a sparse tensor) keyed by the 2D atlas position. Existing `add()` writes to *both* (frame-keyed *and* atlas-keyed) during a transition period; `retrieve()` gains an `use_atlas: bool` flag.

3. New `lyra_2/_src/datasets/uvw_atlas.py`

The bijection helpers. Three functions:

```
def world_to_atlas(world_xyz, world_min, world_max, bits=8):
    """World 3-vec → (atlas_x, atlas_y) 2-vec."""

def atlas_to_world(atlas_xy, world_min, world_max, bits=8):
    """Inverse: (atlas_x, atlas_y) → world 3-vec."""

def quantize_world_to_rgb(world_xyz, world_min, world_max, bits=8):
    """World 3-vec → packed (R, G, B) byte triple. Color IS coord."""
```

All pure-arithmetic, no learnable parameters. We've open-sourced this exact module (MIT) at [voxygen/pipeline/uvw_atlas.py](#) — Lyra-friendly to copy/adapt under Apache-2.0 compatibility.

Bonus: a same-resolution 1-bit occupancy bitmap

Independent of the byte-width family above, a parallel **1-bit-per-voxel companion texture** at the same grid resolution is a low-cost addition worth flagging. It answers a single question — *"is this slot populated?"* — and lets the cross-attention / inference shader **early-skip the multi-byte main-atlas read** for empty voxels.

Cost: 2 MB at 256³ (24× smaller than 3-byte-per-voxel RGB storage, 32× smaller than RGBA8). At 1024³ it's 134 MB.

Three compounding wins:

1. **Storage** — 24× less than 3-byte/voxel
2. **Bandwidth** — same factor; for bandwidth-bound shaders, 10-20× faster scans
3. **Cache locality** — 24× more voxels fit in L1/L2 GPU cache

Pairing with sparse RGB storage (only allocate bytes for populated voxels): dense 1-bit mask gives O(1) addressability + near-hash-table memory cost. For Lyra 2's typical scenes (~5% surface occupancy), the total memory for occupancy + RGB

data drops from 50 MB → ~4.6 MB ^{at 256³}.

← SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

For Lyra 2 specifically: the canonical-coord image has implicit emptiness today (pixels where the warp didn't write content are zeros), but it's a 3-channel zero check — `all(rgb == 0)` — which the cross-attention has to learn to interpret. An explicit 1-bit occupancy channel would:

- Make "is this voxel real or padding?" question structural, not learned
- Cost ~50 KB per inference chunk at Lyra 2's 832 × 480 resolution
- No model retraining needed if exposed as a separate ControlNet input

GLSL is one texel fetch + a shift + an AND:

```
uniform usampler2D occupancyBitmap;    // R8UI, dims atlasW/8 × atlasH
bool is_occupied(ivec2 atlas_xy) {
    uint byte_v = texelFetch(occupancyBitmap,
                            ivec2(atlas_xy.x >> 3, atlas_xy.y), 0).r;
    return ((byte_v >> (atlas_xy.x & 7)) & 1u) != 0u;
}
```

Eight voxels packed per byte along the X-axis preserves cache locality for "scan a row of voxels" access patterns.

What this doesn't claim

- **Quality wins.** I don't have the compute to retrain Lyra 2 and measure SSIM / LPIPS / FID. The argument is architectural: fewer things for the cross-attention to learn, smaller memory footprint for revisiting scenes. Whether that translates to measurable quality improvement is an open empirical question, and entirely depends on retraining infrastructure NVIDIA has and others don't.
- **A drop-in PR.** A retrain is required. This is a proposal to *consider* the encoding shift in a future training run, not a patch.
- **Replacing Sparse3DCache.** The frame-keyed cache has legitimate uses (temporal weighting, multi-view RGB storage). The world-coord atlas could complement it rather than replace it.

Open questions for the team

1. Has the team explored world-coord-keyed conditioning during Lyra 2's design? If yes, what failed?
2. The current `_world_points` storage is already world-coord per pixel — at what cost would the cross-attention be restructured to query that directly?
3. For the long-horizon unbounded case, is a hierarchical sparse atlas competitive with the current frame-keyed approach in terms of memory growth as the camera trajectory length grows?
4. The 8-bit quantization is the simplest version; 16-bit and 32-bit variants are documented in [DownToEarth/README.md](#) → [Scaling section](#). Which precision tier would matter for Lyra 2's training data distribution?

About the proposers

This grew out of building a Lyra-2-inspired local pipeline (voxygen) that runs on consumer GPUs (single iter, ≈ 5 min from a single Juggernaut XL render to a 167 k-Gaussian splat cloud). We can't match Lyra 2's quality — the gap is the 14 B-param Wan-2.1 backbone trained on 64 × GB200s, not the architecture — but the UVW↔RGB bijection landed as a clean little data structure with one property that struck us as worth surfacing upstream: **the bytes are the coords**, both ways, at zero VRAM, and that property is rare enough to be useful beyond our tiny project.

Happy to discuss, refine, or retract any of the above. If the encoding shift turns out to be a bad fit, the comparison itself is still useful data for anyone considering similar architectures.